Practica Final. Diseño de Controladores Borrosos y Neuronales para vehículo en un circuito.

Grado en ingeniería informática

Ainara García García
Omar Eduardo Borges Montero

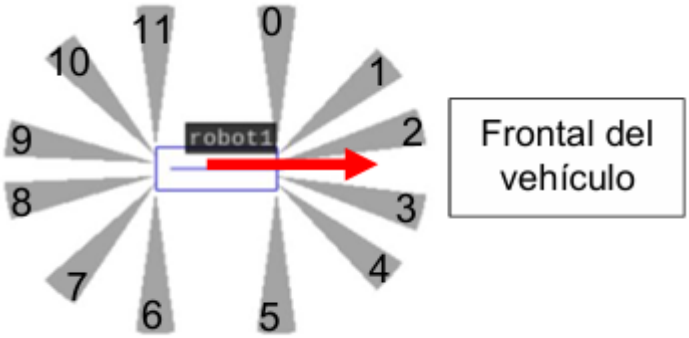
Contenido

Introduccion 3
Parte 1 Control Borroso 4
Sin obstáculos 5
Con obstáculos 10
Parte 2 Control Neuronal 13
Sin obstáculos 13
Entrenamiento sin obstáculos 14
Con obstáculos 18
Entrenamiento con obstáculos 18
Difficultades encontradas 21
Video defensa final parte 2 ..... **¡Error! Marcador no definido.**

Introduccion

El objetivo de esta práctica es diseñar e implementar un sistema de control de velocidad, tanto lineal como angular, para un vehículo, de modo que sea capaz de completar una vuelta a un circuito cerrado de forma eficiente y en el menor tiempo posible.
El entorno de simulación consiste en un circuito ovalado con unas dimensiones totales de 160x80 metros. La pista cuenta con un carril de circulación de 4.5 metros de ancho, y una restricción fundamental es la prohibición de invadir el carril contrario de forma injustificada, permitiéndose únicamente de forma temporal durante las maniobras de adelantamiento.

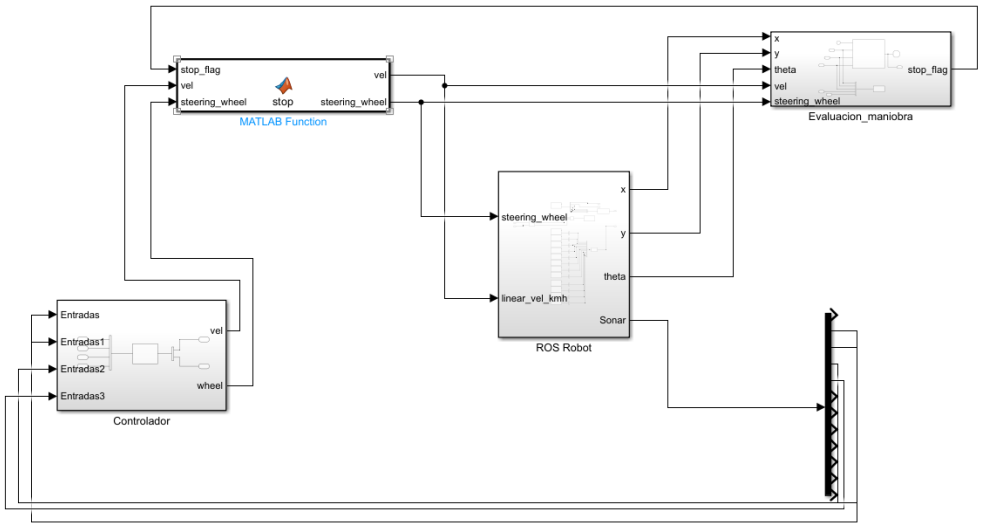
Para desarrollar esta tarea, se utilizarán herramientas como el simulador STDR dentro de ROS (*Robot Operating System*) junto con MATLAB/Simulink, que ofrecen la infraestructura necesaria para crear y probar tanto los controladores borrosos como los neuronales.
El vehículo cuenta con una configuración de tipo Ackermann, lo que implica que no puede moverse únicamente mediante el giro del volante. Siempre requiere una velocidad lineal para desplazarse y no es capaz de modificar su orientación girando sobre su propio eje.
Para percibir el entorno, el vehículo está equipado con 12 sensores ultrasónicos distribuidos alrededor de su chasis, los cuales proporcionan datos sobre la distancia a los muros del circuito y a los obstáculos.



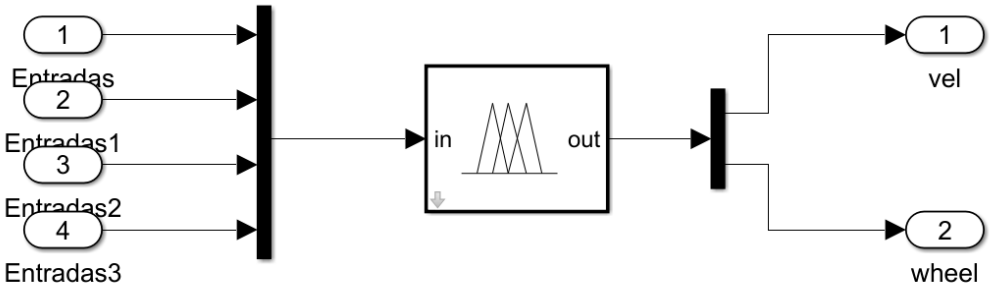
Utilizamos la velocidad lineal y la angular para controlar el coche a lo largo de todo el circuito.
Velocidad lineal: esta siempre es positiva ya que en nuestro circuito el coche nunca puede dar para atrás.
Velocidad angular: es el giro del volante, este valor varia entre -pi/2 y pi/2.

Parte 1 Control Borroso

Para esta primera parte vamos a realizar un controlador borroso de tipo mamdani que se encargue de regular tanto la velocidad lineal como el ángulo de giro del volante para que el coche pueda completar el circuito sin chocarse, ni salirse del carril derecho (siendo los adelantamientos una excepción).
La siguiente imagen representa el esquema de control completo en Simulink:

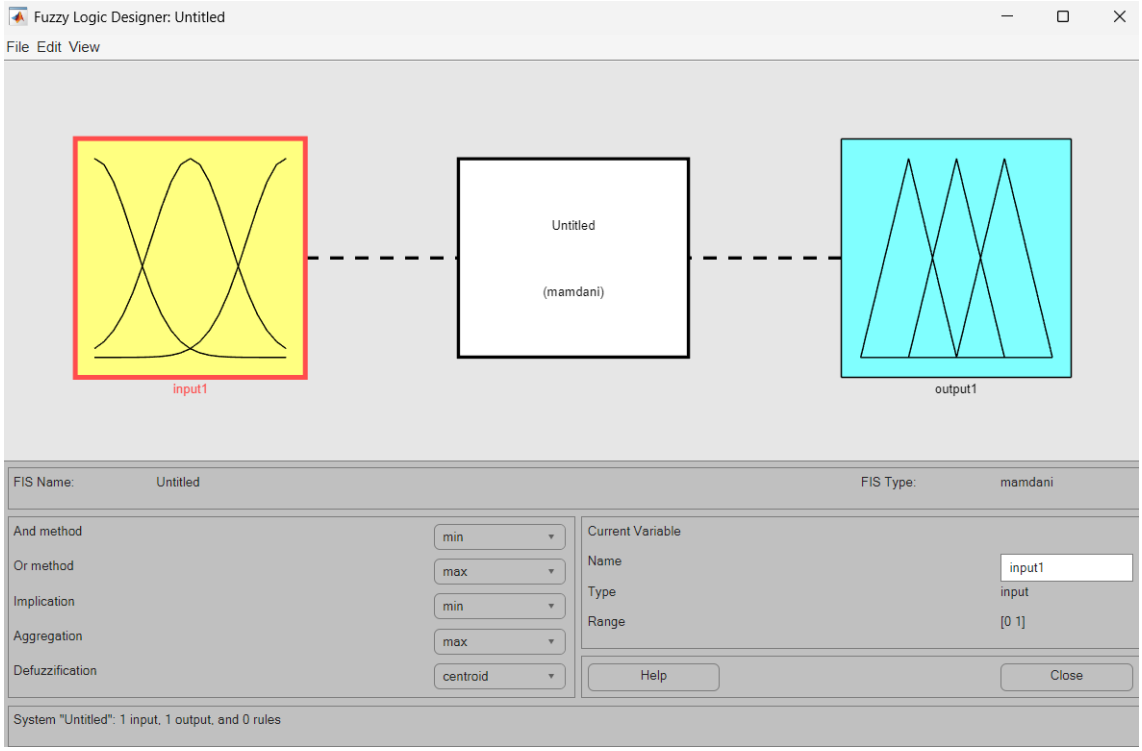


Este seria el esquema que hemos implementado en nuestro controlador:

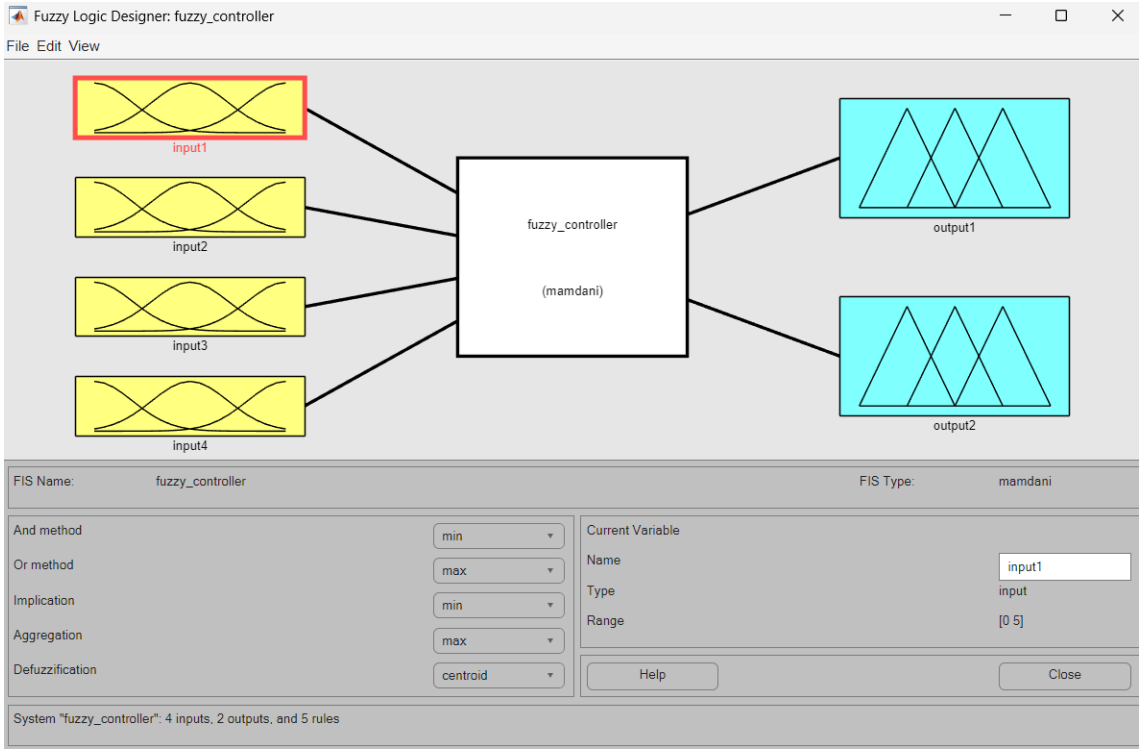


Sin obstáculos

Primero hemos creado el fuzzy_controller, para ello hemos puesto en la consola de Matlab fuzzy y nos aparece la siguiente imagen:



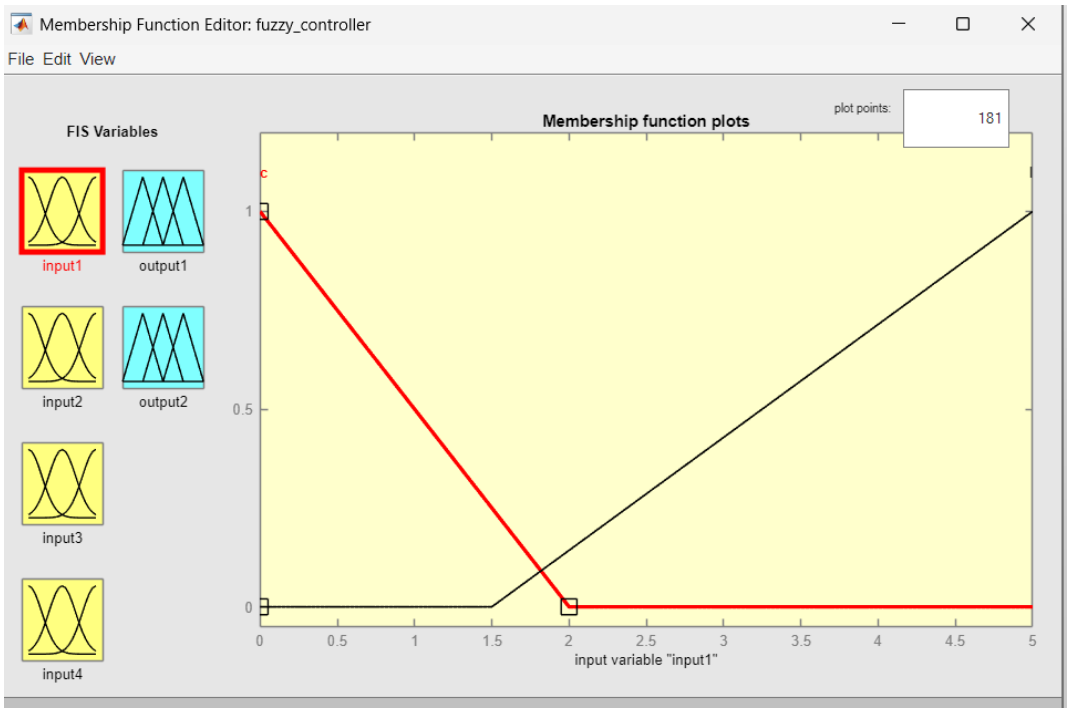
Le dimos a file import y seleccionamos el fuzzy_controler que estaba subido en la carpeta del blackboard y empezamos a ajustar parámetros partiendo de esa base. Lo hemos dividido en variables de entrada (inputs) y variables de salida (outputs).



En cuanto a los inputs hemos utilizado los sonars 1,2,3,4 y todos utilizan funciones de pertenencia triangulares del tipo trimf:

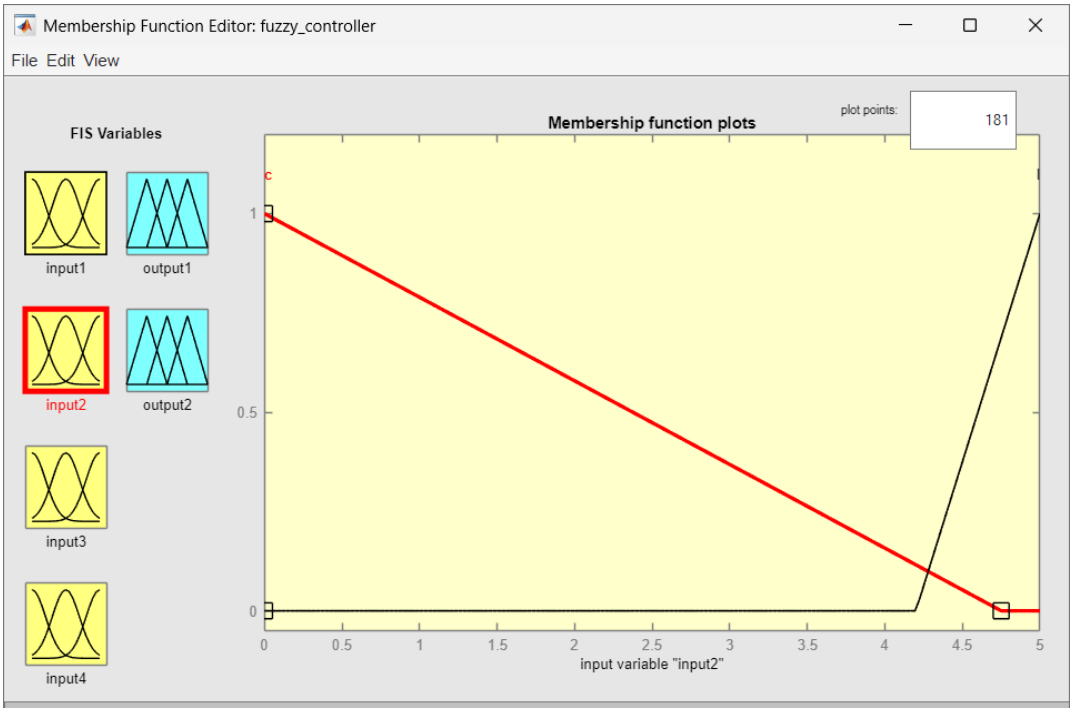
El sonar 1 cuenta con las siguientes funciones de pertenencia:

- c: que seria cerca y emplea los valores [0 0 2]
- l :que seria lejos y emplea los valores [1.5 5 5]



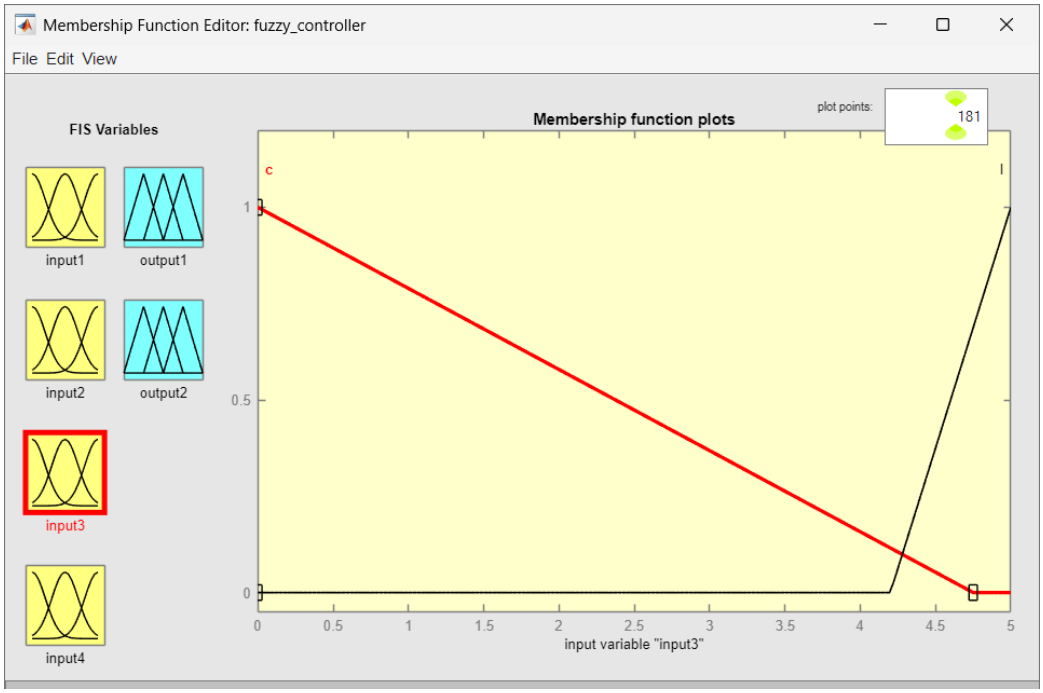
El sonar 2 cuenta con las siguientes funciones de pertenencia:

- c: que seria cerca y emplea los valores [0 0 4.75]
- l :que seria lejos y emplea los valores [4.2 5 5]



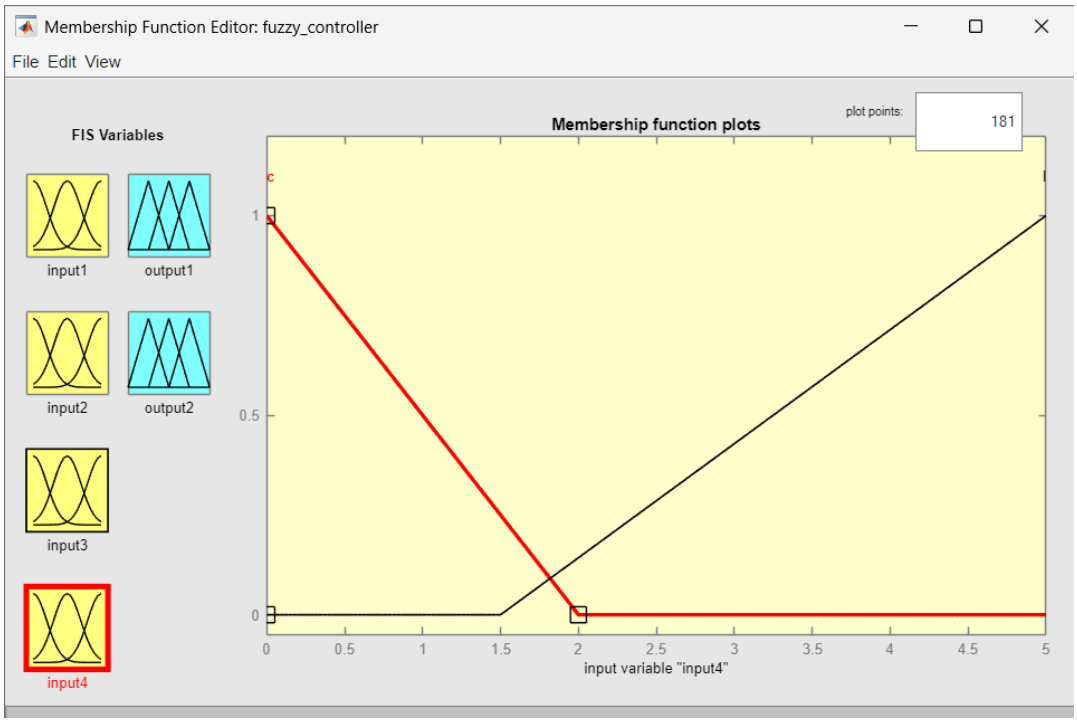
El sonar 3 cuenta con las siguientes funciones de pertenencia:

- c: que seria cerca y emplea los valores [0 0 4.75]
- l :que seria lejos y emplea los valores [4.2 5 5]



El sonar 4 cuenta con las siguientes funciones de pertenencia:

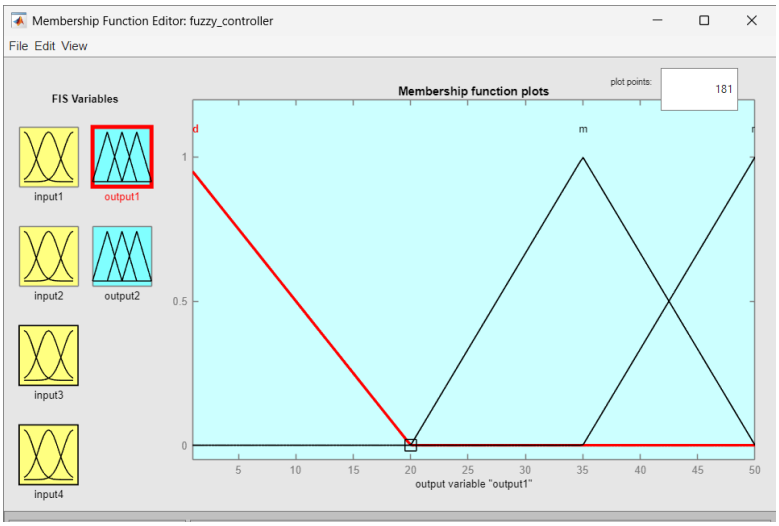
- c: que seria cerca y emplea los valores [0 0 2]
- l :que seria lejos y emplea los valores [1.5 5 5]



En cuanto a las variables de salida tenemos el output1 que es la velocidad lineal y el output2 que es el ángulo de giro del robot:

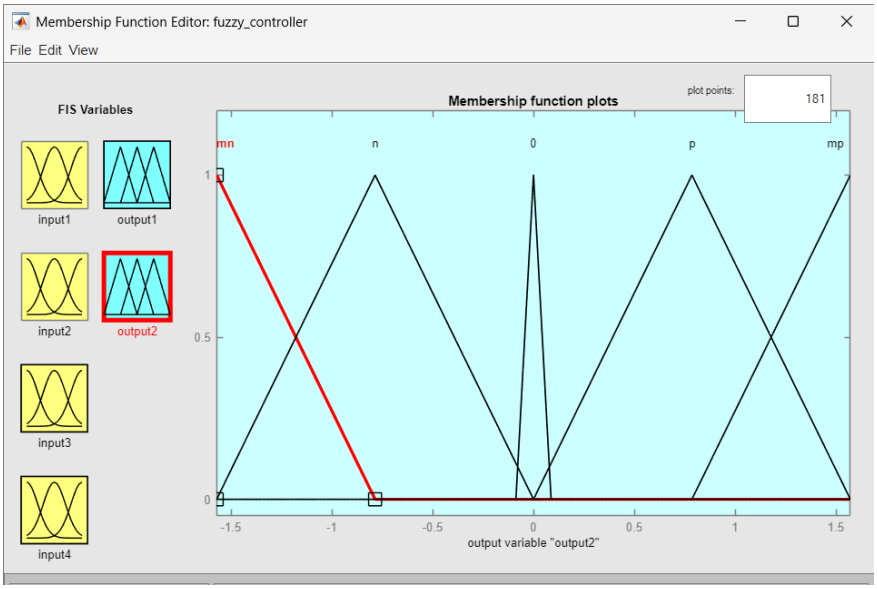
El output1 cuenta con las siguientes funciones de pertenencia:

- d: que sería despacio y emplea los valores [0 0 20]
- m: que sería medio y emplea los valores [20 35 50]
- r: que sería rapido y emplea los valores [35 50 50]
-



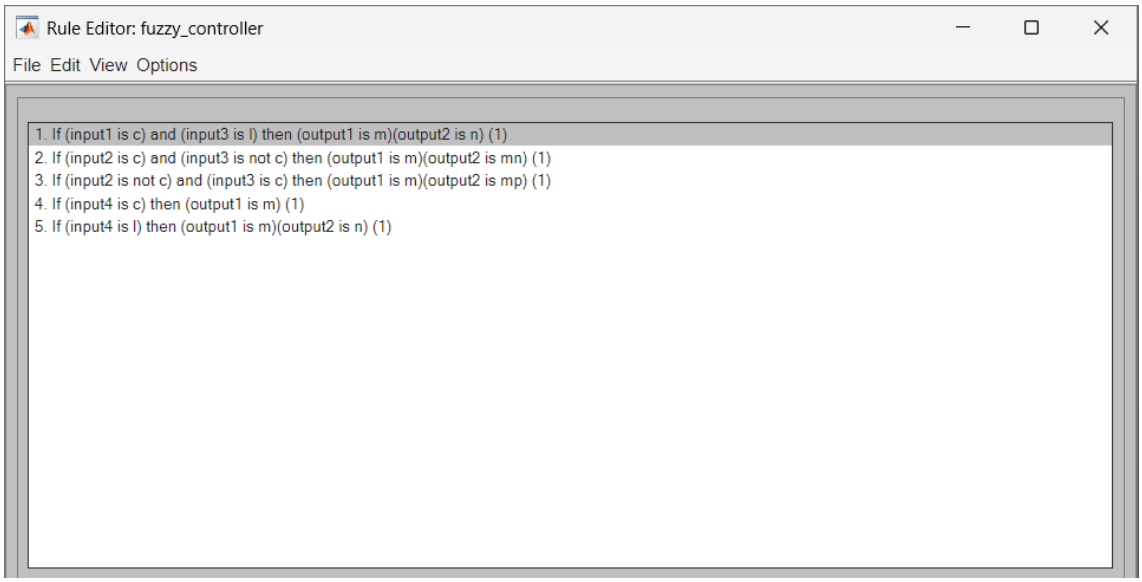
El output2 cuenta con las siguientes funciones de pertenencia:

- mn: que sería muy negativo y emplea los valores [-1.571 -1.571 -0.7854]
- n: que sería negativo y emplea los valores [-1.57 -0.785 0]
- 0: que sería cero y emplea los valores [-0.087 0 0.087]
- p: que sería positivo y emplea los valores [0 0.785 1.57]
- mp: que sería muy positivo y emplea los valores [0.785 1.57 1.57]



Ángulos positivos (mp y p) → Giro a la izquierda
Angulo 0 (0)→ continua recto
Ángulos negativos(mn y n) → Giro a la derecha

Para que el robot pueda moverse de manera eficiente y seguir el circuito correctamente sin salirse del carril derecho, es necesario definir un conjunto de reglas difusas que relacionen la posición o ángulo del robot con la acción de giro que debe realizar. Estas reglas permiten al controlador Mamdani tomar decisiones, facilitando movimientos suaves y precisos.
Las reglas que hemos definido son las siguientes:



Regla 1: If (input1 is c) and (input3 is l) then (output1 is m)(output2 is n) (1)
Esta regla se encarga de monitorizar la proximidad de la pared izquierda a través del input1. Si este sensor detecta un valor cercano (c) y el sensor frontal-derecho (input3) está libre (l), el robot realiza un giro suave a la derecha (n) para corregir su trayectoria y separarse del muro, manteniendo siempre la velocidad de crucero de 35 km/h (m).

Regla 2 : If (input2 is c) and (input3 is not c) then (output1 is m)(output2 is mn) (1)
Esta actúa como una medida de seguridad ante colisiones frontales por el lado izquierdo. Si el input2 detecta un obstáculo cercano (c) y el input3 no está bloqueado (not c), el controlador ordena un giro de emergencia máximo a la derecha (mn). Al igual que en el resto de la lógica, se asegura de que el vehículo no se detenga manteniendo la velocidad en el valor medio (m).
Regla 3: If (input2 is not c) and (input3 is c) then (output1 is m)(output2 is mp) (1)
Gestiona los obstáculos detectados en el flanco frontal-derecho a través del input3. Cuando este sensor detecta un objeto cercano y el lado izquierdo está despejado (input2 es not c), el robot ejecuta un giro rápido hacia la izquierda (mp) a una velocidad constante de 35 km/h para evitar el impacto.
Regla 4: If (input4 is c) then (output1 is m) (1)
La hemos establecido para garantizar que el coche mantenga su velocidad de crucero de forma constante siempre que circule paralelo a la pared. Siempre que el sensor derecho (input4) detecte la proximidad del muro (c), el sistema fuerza la salida de velocidad a su valor medio (m), pero dejamos la dirección sin definir para evitar conflictos con otras maniobras de giro.
Regla 5: If (input4 is l) then (output1 is m)(output2 is n) (1)
Sirve como sistema de rastreo o "seguimiento de carril". Si el sensor lateral derecho detecta que la distancia a la pared es grande (l), significa que el coche se está alejando de

su referencia; por tanto, el robot corrige su posición girando suavemente a la derecha (n) hasta recuperar la proximidad deseada, manteniendo la estabilidad en la velocidad de 35 km/h.

Después de hacer un montón de pruebas en el simulador, nos dimos cuenta de que mantener una velocidad constante de 35 km/h era la clave para conseguir el tiempo más bajo en el circuito. Aunque la lógica nos decía que podíamos correr más en las rectas, el problema era que si acelerábamos demasiado, el coche llegaba con muchísima inercia a la entrada de la siguiente curva y, como nuestro volante tiene un giro máximo de 90 grados, el coche no tenía capacidad física para trazar el giro y se acababa estampando contra el muro. Además, los sensores del coche ven a una distancia máxima de unos 5 metros, y a una velocidad mayor de 35 km/h, el sistema no tenía tiempo suficiente para procesar la información y ejecutar la maniobra de esquiva antes del impacto. Al final, mantener esos 35 km/h constantes durante todo el trayecto (configurando el Output1 en el rango de [1 50] con la función MF2 centrada exactamente en 35) resultó ser lo más eficiente.

Video borroso sin obstaculos

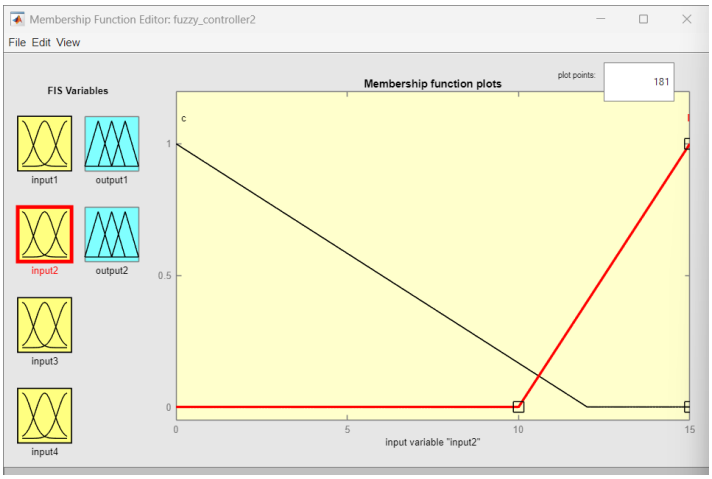
Adjuntamos un video en el que se demuestra el funcionamiento de esta parte:
https://drive.google.com/file/d/1AAivnFQG68yXVfA4AxFIGbbFPGfpuKlO/view?usp=sharing

Con obstáculos

En cuanto al circuito con obstáculos hemos utilizado el mismo modelo que simulink pero hemos hecho algunos cambios en el fuzzy_controller que ahora explicaremos.

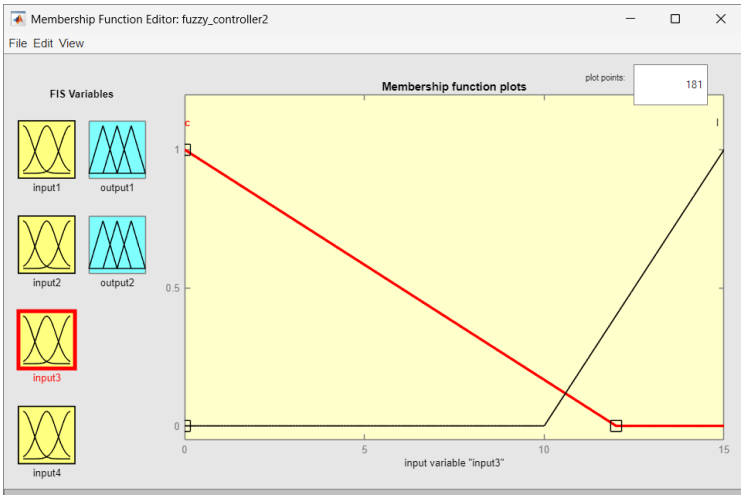
La primera modificación que hemos realizado es cambiar los rangos de los inputs 2 y 3 para que puedan detectar los obstáculos a 15m ya que 5 era muy poca distancia.
El sonar 2 cuenta con las siguientes funciones de pertenencia:

- c: que sería cerca y emplea los valores [0 0 12]
- l :que sería lejos y emplea los valores [10 15 15]



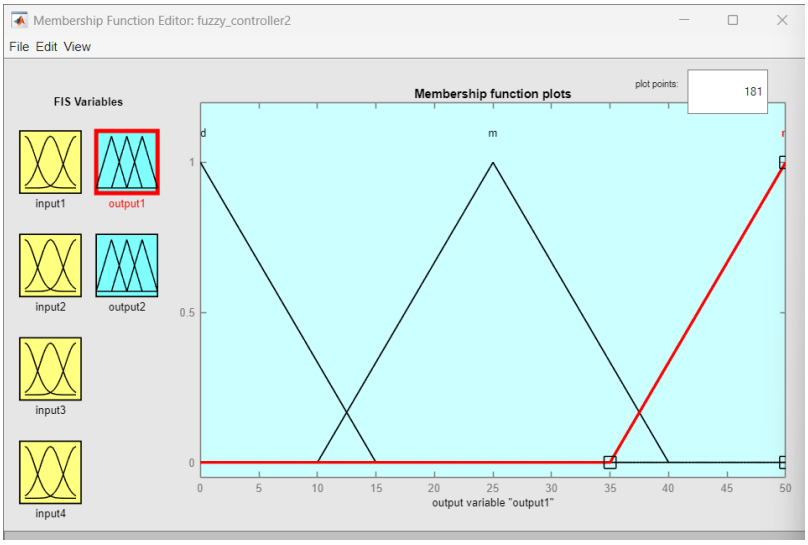
El sonar 2 cuenta con las siguientes funciones de pertenencia:

- c: que sería cerca y emplea los valores [0 0 12]
- l :que sería lejos y emplea los valores [10 15 15]
-

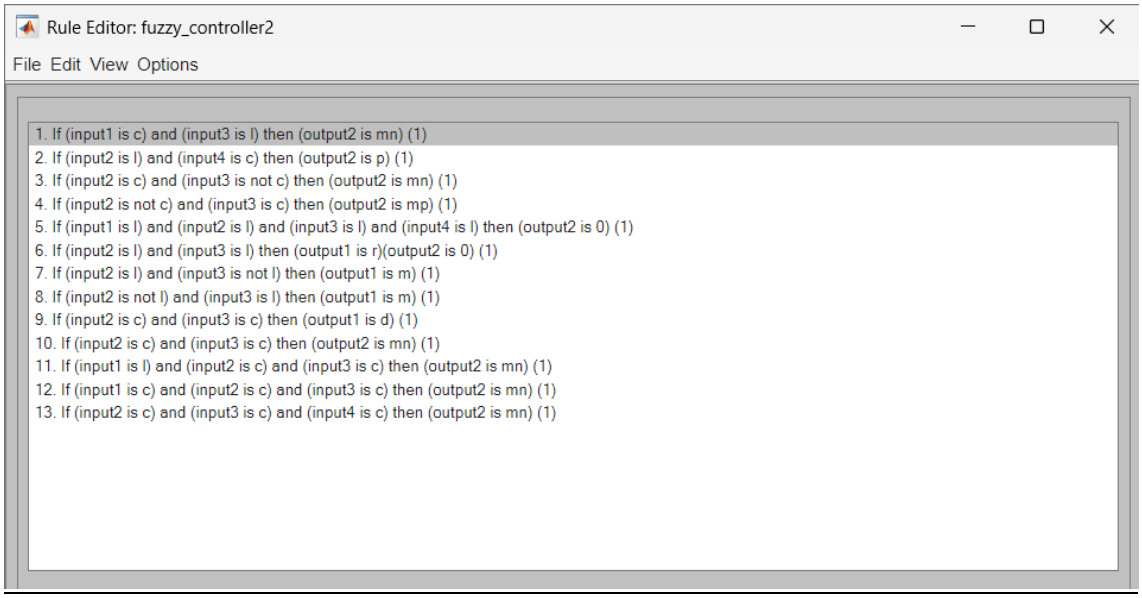


Respecto a los outputs hemos disminuido la velocidad media y despacio ya que al ir tan rápido no le daba tiempo a esquivar los obstáculos y se chocaba:
El output1 cuenta con las siguientes funciones de pertenencia:

- d: que sería despacio y emplea los valores [0 0 15]
- m: que sería medio y emplea los valores [10 25 40]
-



Por ultimo al incorporar obstáculos hemos tenido que añadir reglas para que nuestro coche sea capaz de esquivarlos ya que las anteriores solo servían para que no se chocase y no se saliese del carril.



Las reglas que hemos añadido son las siguientes:

- **Regla 2:** permite al robot mantenerse alineado con el carril. Si el frente izquierdo está libre y la pared derecha está cerca, el robot gira a la izquierda (p) para no pegarse demasiado al borde derecho.
- **Regla 5**: cuando los cuatro sensores detectan distancias grandes (l), el controlador clava el volante en la posición neutra (0) para avanzar en línea recta sin oscilaciones.
- **Regla 6:** complementa la anterior en las rectas despejadas, asegurandonos que el volante se mantenga recto (0) siempre que los dos sensores frontales no vean obstáculos, tambien permite que el robot alcance su rendimiento máximo. Si el camino frontal está totalmente despejado, el controlador asigna la velocidad rápida (r) para reducir el tiempo de vuelta.
- **Reglas 7 y 8:** gestionan la velocidad en situaciones de tráfico o curvas medias. Si uno de los sensores frontales detecta que el camino se estrecha o no está totalmente libre, el coche reduce automáticamente a velocidad media (m) para ganar control.
- **Regla 9** funciona como un freno de seguridad. Cuando los dos sensores frontales detectan una pared u obstáculo muy cerca (c), la velocidad cae al valor mínimo (d) para evitar un impacto violento y permitir que las reglas de giro tengan tiempo de actuar.
- **Reglas 10, 11, 12 y 13** establecen el protocolo de emergencia si ambos sensores frontales detectan una pared de frente, o si se combinan obstáculos frontales con laterales, el sistema prioriza un giro de escape hacia la derecha (mn) para intentar desbloquear la situación.

Video borroso con obstáculos

Adjuntamos un video en el que se demuestra el funcionamiento de esta parte:
https://drive.google.com/file/d/1uXe93IycUv5Awq-vifpUfayx9JNhVnjV/view?usp=sharing

Parte 2 Control Neuronal

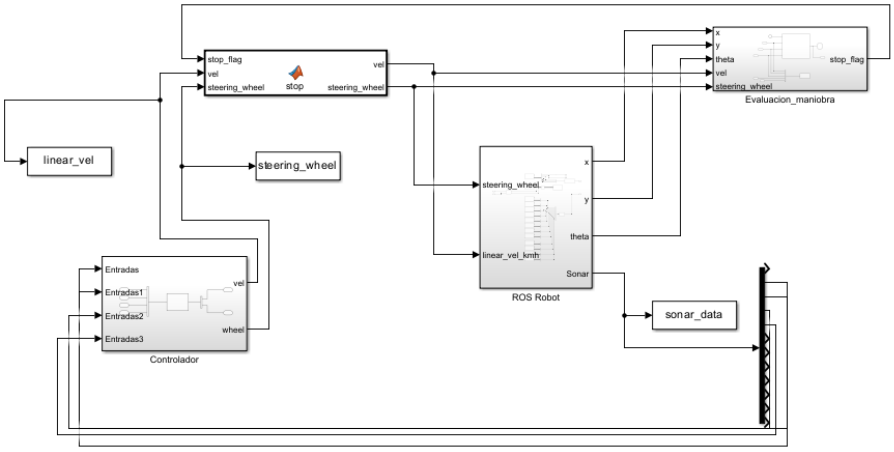
Sin obstáculos

Para el diseño del controlador neuroborroso es necesario, en primer lugar, disponer de un conjunto de datos suficientemente amplio y representativo para el entrenamiento de la red neuronal. Inicialmente, se realizaron varios intentos de generación de datos mediante el script de telecontrol manual proporcionado. Sin embargo, este método resultó poco eficiente para la obtención de una gran cantidad de muestras continuas y estables en el tiempo.

Por este motivo, se decidió emplear el controlador borroso previamente diseñado y ajustado por mi compañera, el cual ofrecía un comportamiento robusto y estable del vehículo. Dicho controlador permitía completar el circuito con una velocidad media, obteniendo un tiempo aproximado de 1 minuto y 20 segundos por vuelta, lo que lo hacía especialmente adecuado para la generación automática de datos de entrenamiento.

Utilizando este controlador borroso, el vehículo fue simulado durante 3 vueltas del circuito, con un TimeStep de 0.1 segundos, lo que permitió registrar aproximadamente 2440 muestras de datos. En cada instante se almacenaron las lecturas de los sensores, la posición, la orientación, así como las velocidades lineal y angular del vehículo, generando así una base de datos extensa y homogénea, especialmente adecuada para el entrenamiento del modelo neuroborroso.

La gran cantidad de datos obtenidos mediante este procedimiento ha resultado fundamental para mejorar la calidad del entrenamiento, ya que permite capturar de forma más precisa la dinámica del sistema y el comportamiento del vehículo a lo largo de todo el circuito, tanto en tramos rectos como en curvas.



Todos los datos necesarios fueron extraídos mediante los bloques To Workspace de Simulink, los cuales se colocaron en las salidas correspondientes a Sonar, Vel y Steering wheel, permitiendo así el almacenamiento y posterior procesamiento de las variables requeridas para el entrenamiento del modelo.

Entrenamiento sin obstáculos

Para el circuito sin obstáculos se ha desarrollado un script en Matlab que permite ejecutar varias simulaciones consecutivas y recopilar automáticamente los datos necesarios para el entrenamiento del controlador neuronal.

En cada simulación se lanza el modelo de Simulink, controlado mediante el controlador borroso previamente validado, y se almacenan las lecturas de los sensores y las acciones de control generadas. Con el objetivo de aumentar la cantidad y diversidad de los datos, se realizaron tres ejecuciones completas del circuito, acumulando las muestras obtenidas en cada una de ellas.

Para el conjunto de entradas del modelo, se han seleccionado los sensores ultrasónicos utilizados en el controlador borroso, concretamente los sónares frontales y laterales, correspondientes a los índices 0, 1, 2 y 3, que proporcionan información suficiente sobre el entorno inmediato del vehículo en ausencia de obstáculos.

En cuanto a las salidas, se han empleado las variables de velocidad lineal y ángulo del volante (steering wheel), que representan las acciones de control que el modelo neuronal debe aprender a reproducir.

Durante el proceso de recopilación de datos, se llevó a cabo una fase de preprocesado, sustituyendo valores infinitos o no definidos por valores máximos del rango admisible, con el fin de evitar problemas durante el entrenamiento de la red neuronal. Una vez finalizadas las simulaciones, los datos se agruparon y reorganizaron en el formato requerido por la Neural Network Toolbox de Matlab, y se almacenaron en un archivo para su posterior uso en el entrenamiento.

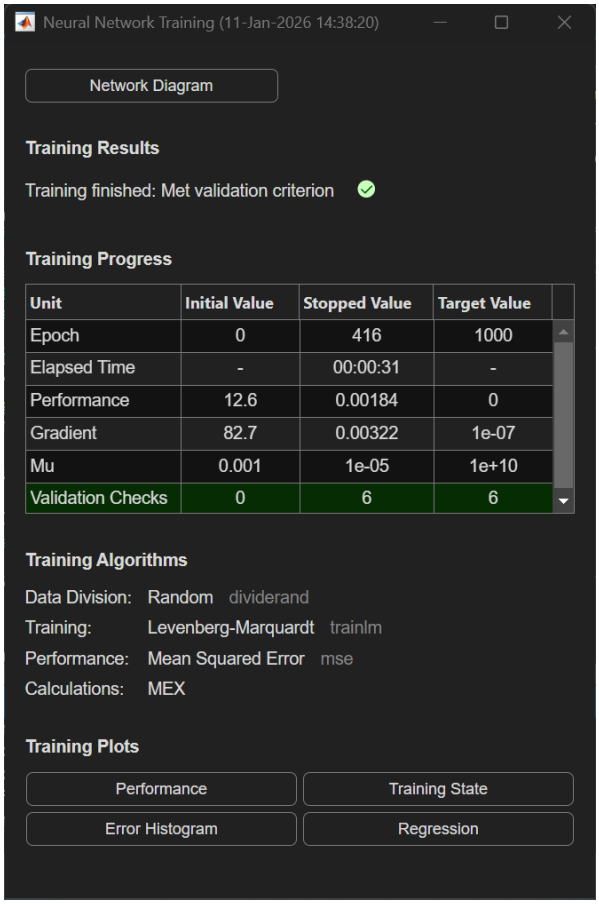
```
clc;
clear;
close all;

% Número de simulaciones (vueltas)
num_runs = 3;

% Dataset final
inputs_all  = [];
outputs_all = [];

for i = 1:num_runs
    disp(['Ejecución de la simulación ', num2str(i)]);

    % Iniciar simulación en Simulink
    sim('ackerman_ROS_controller');   % <-- nombre del modelo

    % --- ENTRADAS ---
    % Seleccionar ÚNICAMENTE los sensores utilizados en el controlador
borroso
    inputs = [
        sonar_data(:,1), ...
        sonar_data(:,2), ...
    sonar_data(:,3), ...
    sonar_data(:,4)
```

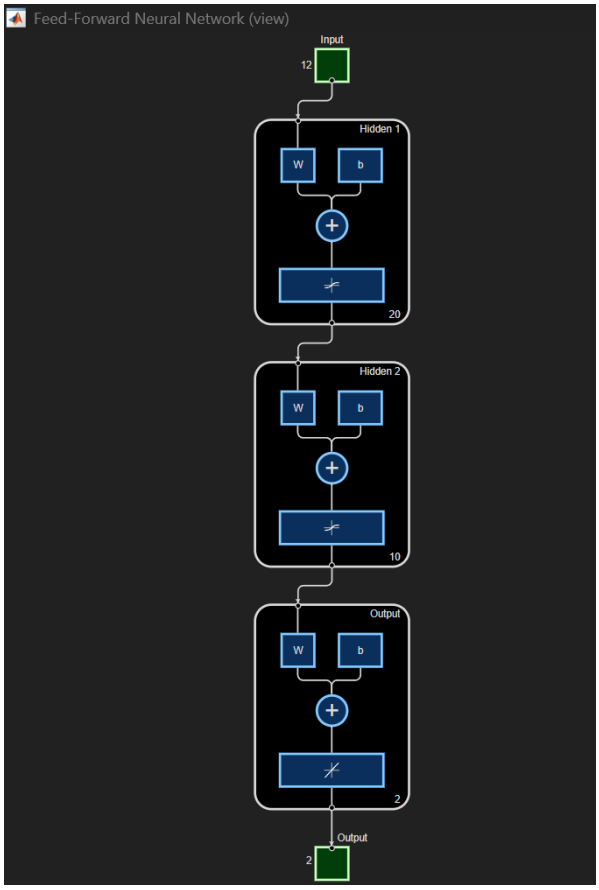
```
    ];

    % --- SALIDAS ---
    outputs = [
        linear_vel, ...
        steering_wheel
    ];

    % Limpieza de datos
    inputs(isinf(inputs)) = 5;
    inputs(isnan(inputs)) = 5;

    % Acumulación de datos
    inputs_all  = [inputs_all; inputs];
    outputs_all = [outputs_all; outputs];
end

% Transposición para Neural Network Toolbox
inputs_all  = inputs_all';
outputs_all = outputs_all';

% Guardado del dataset
save('training_data.mat','inputs_all','outputs_all');

disp('Dataset de entrenamiento guardado correctamente');
```

Los datos utilizados para el entrenamiento se obtuvieron directamente desde Simulink mediante bloques To Workspace, conectados a las salidas de los sensores ultrasónicos, la velocidad lineal y el ángulo del volante. Estas señales se almacenaron en variables de Matlab, que posteriormente fueron procesadas mediante el script de generación del dataset.

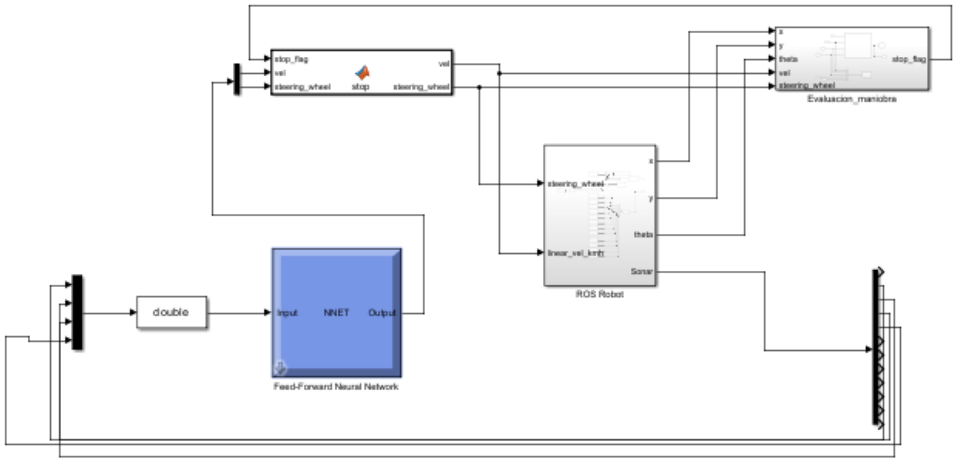
Las matrices de entrada y salida se construyeron concatenando los datos de todas las simulaciones realizadas, obteniendo así un conjunto de entrenamiento más amplio y representativo del comportamiento del vehículo a lo largo del circuito. Posteriormente, las matrices se transpusieron para adaptarse al formato requerido por las herramientas de entrenamiento neuronal de Matlab, donde cada columna representa una muestra independiente.
Una vez preparado el conjunto de datos, se procedió al entrenamiento de una red neuronal de tipo feedforward, cuyo objetivo es aproximar el comportamiento del controlador borroso utilizado durante la fase de generación de datos.

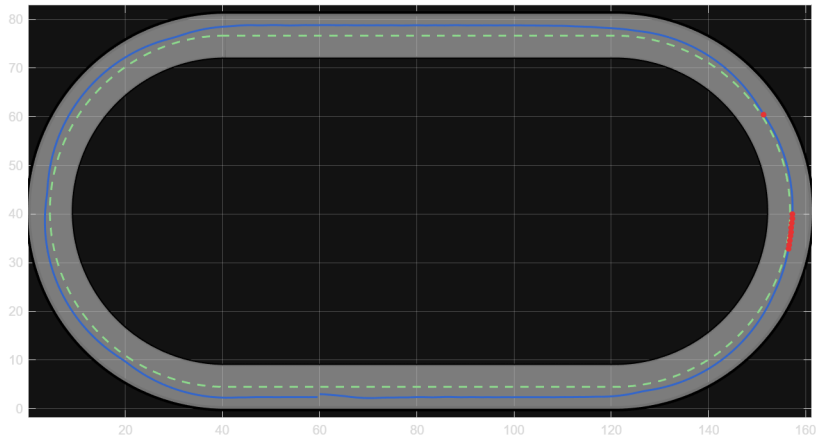
La red neuronal se configuró utilizando como entradas las lecturas de los sensores seleccionados y como salidas la velocidad lineal y el ángulo del volante. Tras definir la arquitectura de la red y sus parámetros de entrenamiento, se llevó a cabo el proceso de aprendizaje mediante el conjunto de datos previamente generado.

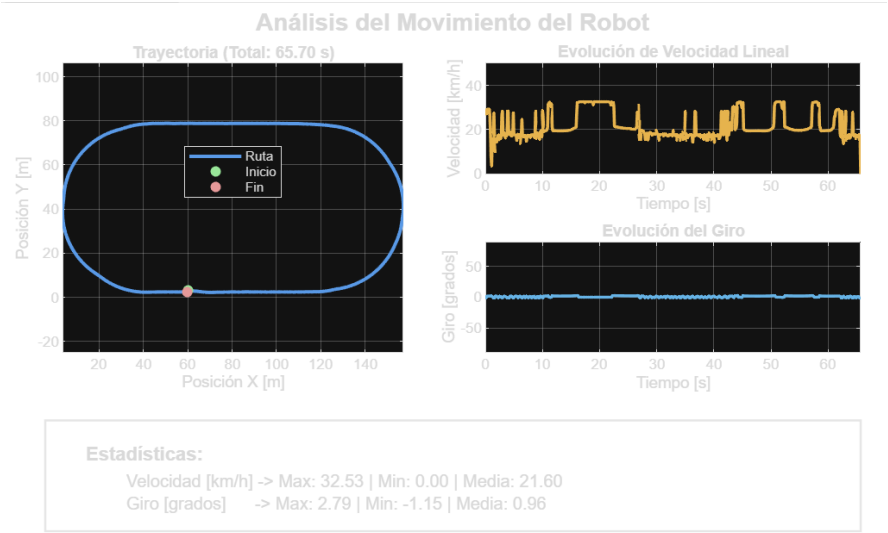

Neural Network Training (11-Jan-2026 14:38:20)
Network Diagram
Training Results
Training finished: Met validation criterion
Training Progress
Unit | Initial Value | Stopped Value | Target Value
Epoch | 0 | 416 | 1000
Elapsed Time | - | 00:00:31 | -
Performance | 12.6 | 0.00184 | 0
Gradient | 82.7 | 0.00322 | 1e-07
Mu | 0.001 | 1e-05 | 1e+10
Validation Checks | 0 | 6 | 6
Training Algorithms
Data Division: Random dividerand
Training: Levenberg-Marquardt trainlm
Performance: Mean Squared Error mse
Calculations: MEX
Training Plots
Performance
Training State
Error Histogram
Regression


Feed-Forward Neural Network (view)
Input
12
Hidden 1
W
b
20
Hidden 2
W
b
10
Output
W
b
2
Output
2

Finalizado el entrenamiento, el modelo neuronal resultante se exportó y se generó automáticamente el correspondiente bloque de Simulink, lo que permitió integrar el controlador neuronal en el esquema de simulación y evaluar su comportamiento en el entorno virtual, sustituyendo al controlador borroso original.







Video neuronal sin obstáculos

Adjuntamos un video en el que se demuestra el funcionamiento de esta parte: https://drive.google.com/file/d/1sCnwFRKqWydNu_qUIbZRTTuEMR7oY45t/view?usp=drive_link

Con obstáculos

Para el circuito con obstáculos se ha seguido un procedimiento análogo al descrito en el caso sin obstáculos, con el objetivo de generar un conjunto de datos para el entrenamiento de un controlador neuronal. En este caso, el recorrido se realizó utilizando directamente el controlador borroso diseñado para el entorno con obstáculos, el cual se encargaba del control completo del vehículo durante la simulación.
Al igual que en el caso anterior, los datos fueron extraídos mediante bloques To Workspace situados en las salidas de los sensores ultrasónicos, la velocidad lineal y el ángulo del volante, permitiendo su posterior tratamiento en el entorno de Matlab. Se realizaron varios intentos de simulación, empleando diferentes configuraciones y estrategias de control, con el fin de capturar el comportamiento del vehículo en presencia de obstáculos.

Para la selección de las entradas del modelo, se utilizaron los sensores frontales y laterales, de manera que el controlador neuronal dispusiera de información suficiente sobre la proximidad tanto de los muros del circuito como de los obstáculos situados en la calzada. Las salidas del modelo se definieron, de nuevo, como la velocidad lineal y el ángulo del volante.

Entrenamiento con obstáculos

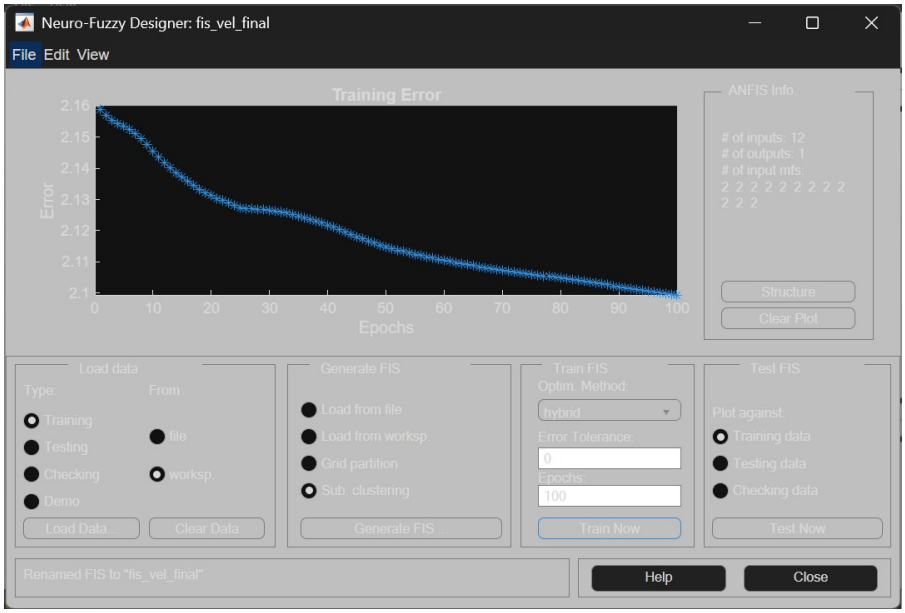
Durante el proceso de generación de datos se siguió el mismo procedimiento aplicado en el caso sin obstáculos, incluyendo las mismas técnicas de preprocesado, sustituyendo valores infinitos o no definidos por valores dentro del rango admisible. A partir de estos datos, se llevaron a cabo distintos intentos de entrenamiento del controlador, optándose finalmente por el uso de la herramienta AnfisEdit, al ser el enfoque que ofreció los resultados relativamente más estables.

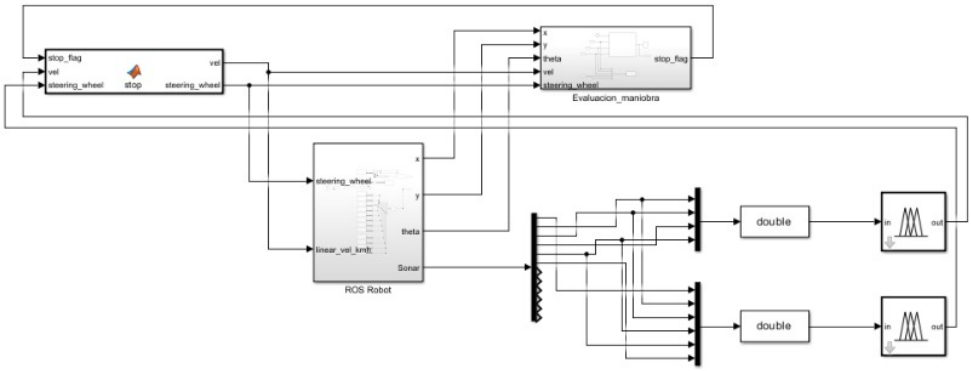
Además, se decidió diferenciar los métodos empleados en los dos escenarios no solo por la mayor complejidad del entorno con obstáculos, sino también con el objetivo de aprender y profundizar en nuevas funcionalidades de las herramientas disponibles en Matlab para el diseño de controladores neuroborrosos. Esta elección permitió ampliar la comprensión práctica del proceso de entrenamiento automático de sistemas borrosos, incluso aunque el resultado final no alcanzara un comportamiento completamente satisfactorio.

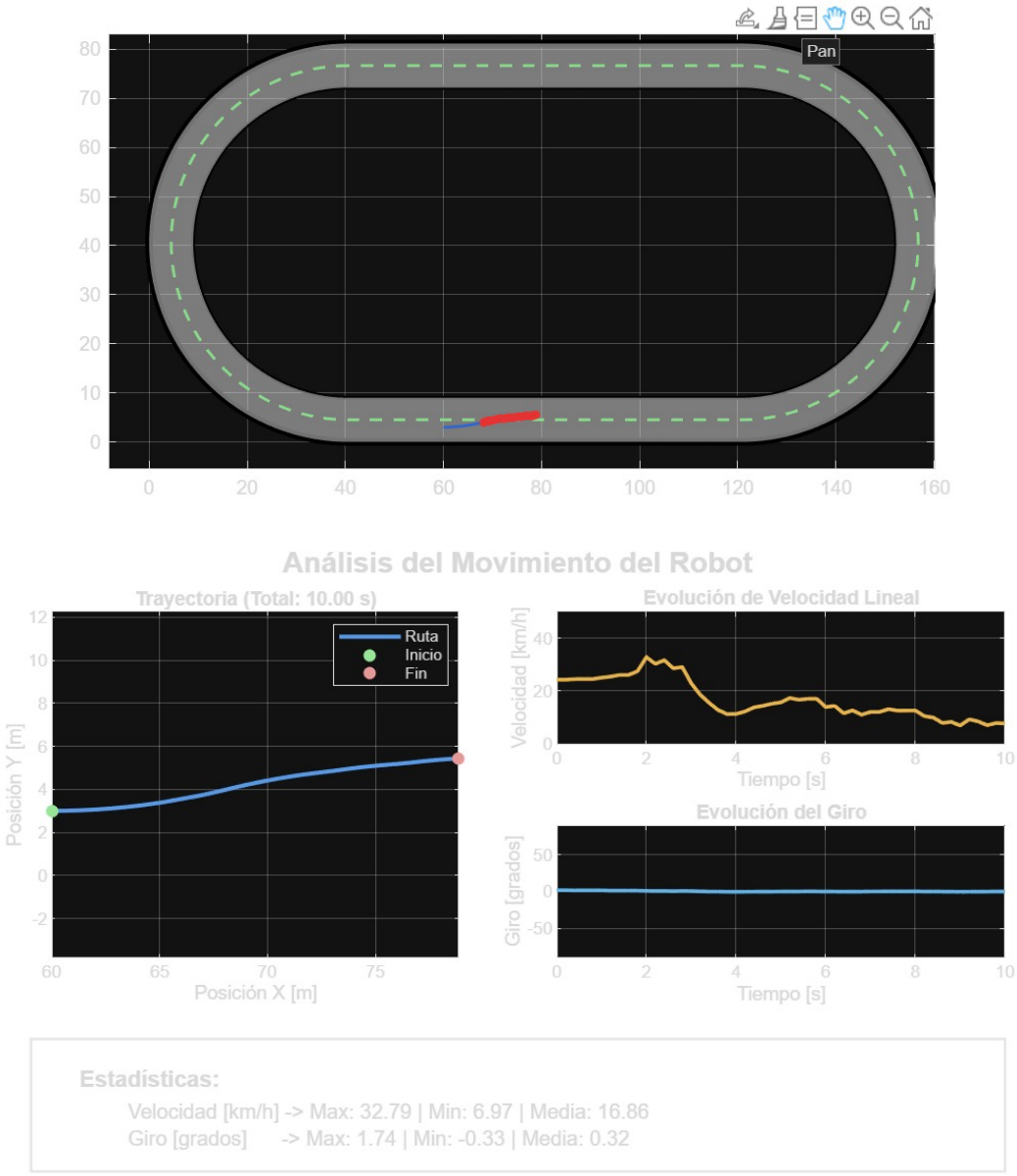
No obstante, debido a la inestabilidad del comportamiento del vehículo en el entorno con obstáculos, los datos obtenidos resultaron ser limitados y poco consistentes, lo que dificultó el proceso de entrenamiento y la capacidad de generalización del modelo. Como consecuencia, aunque el controlador neuroborroso generado mediante AnfisEdit mostró un comportamiento ligeramente mejor en comparación con otras estrategias probadas, el vehículo no logró completar el circuito, colisionando en la mayoría de los casos, tras recorrer una corta distancia, contra los obstáculos o contra los muros del circuito.

```
% Convertimos los datos a 'double' para evitar errores de precisión en
ANFIS
S = double(sonar_data);
V = double(linear_vel);
G = double(steering_wheel);

% Eliminamos datos donde el robot está estático
idx_en_movimiento = V > 0.01;
S = S(idx_en_movimiento, :);
V = V(idx_en_movimiento, :);
G = G(idx_en_movimiento, :);

% Dataset para VELOCIDAD (Entradas: sensores frontales 1, 2, 3, 4 ->
Columnas 2,3,4,5)
% Formato ANFIS requerido: [Entradas, Salida]
dataset_velocidad = [S(:, 2:5), V];

% Dataset para DIRECCIÓN/GIRO (Entradas: sensores 0, 1, 2, 3, 4, 5 ->
Columnas 1,2,3,4,5,6)
dataset_direccion = [S(:, 1:6), G];

anfisedit
```

Neuro-Fuzzy Designer: fis_sterzo_final
File Edit View
Training Error
Error
Epochs
ANFIS Info.
# of inputs: 12
# of outputs: 1
# of input mfs:
2 2 2 2 2 2 2 2
2 2 2
Structure
Clear Plot
Load data
Type:
From:
Training
Testing
Checking
Demo
file
worksp.
Load Data...
Clear Data
Generate FIS
Load from file
Load from worksp.
Grid partition
Sub. clustering
Generate FIS ...
Train FIS
Optim. Method:
hybrid
Error Tolerance:
0
Epochs:
100
Train Now
Test FIS
Plot against:
Training data
Testing data
Checking data
Test Now
Renamed FIS to "fis_sterzo_final"
Help
Close


Neuro-Fuzzy Designer: fis_vel_final
File Edit View
Training Error
Error
Epochs
ANFIS Info.
# of inputs: 12
# of outputs: 1
# of input mfs:
2 2 2 2 2 2 2 2
2 2 2
Structure
Clear Plot
Load data
Type:
From:
Training
Testing
Checking
Demo
file
worksp.
Load Data...
Clear Data
Generate FIS
Load from file
Load from worksp.
Grid partition
Sub. clustering
Generate FIS ...
Train FIS
Optim. Method:
hybrid
Error Tolerance:
0
Epochs:
100
Train Now
Test FIS
Plot against:
Training data
Testing data
Checking data
Test Now
Renamed FIS to "fis_vel_final"
Help
Close


stop_flag
vel
steering_wheel
stop
Evaluacion_maniobra
ROS Robot
double
double



Como resultado, no fue posible obtener un controlador neuronal completamente funcional para el entorno con obstáculos. Sin embargo, el trabajo realizado ha permitido identificar claramente las limitaciones del enfoque empleado, así como evidenciar la mayor dificultad inherente al aprendizaje automático en entornos no estructurados y dinámicos, donde pequeñas variaciones en las entradas sensoriales pueden provocar comportamientos no deseados o inestables en la salida del sistema.

SCI-neuralobs.mp4

Difficultades encontradas

A diferencia del entorno sin obstáculos, el escenario con obstáculos presentó una complejidad considerablemente mayor, debido a la presencia de elementos dinámicos que requieren una capacidad de reacción más precisa y robusta por parte del controlador. A lo largo del desarrollo se probaron diversas estrategias de control, incluyendo modificaciones en la selección de los sensores utilizados, ajustes en las variables de entrada y salida, así como cambios en la estrategia de generación y entrenamiento de los datos.

No obstante, en todos los enfoques evaluados el comportamiento del vehículo resultó inestable. En la mayoría de las simulaciones, el vehículo lograba iniciar la marcha de forma correcta, pero tras recorrer una distancia reducida terminaba colisionando, bien contra alguno de los obstáculos presentes en el circuito, bien contra los muros laterales del carril. Este comportamiento impedía completar trayectorias suficientemente largas y coherentes como para generar un conjunto de datos representativo y, en consecuencia, entrenar de manera eficaz un controlador neuronal con capacidad de generalización.